



探索性数据分析与可视化技术课程报告

OpenSourceLens

基于 GitHub 开源项目数据的 交互式探索性分析与可视化系统

课程名称：探索性数据分析与可视化技术
学生姓名：
学号：
指导老师：王瀚漓
完成时间：2026 年 6 月

OpenSourceLens：基于 GitHub 开源项目数据的交互式探索性分析与可视化系统

摘要

开源软件生态的规模快速扩大，单一的 Stars 指标已经难以完整刻画项目价值、维护状态和社区活力。本文设计并实现了 OpenSourceLens，一个基于 Reflex 的中文交互式数据可视化平台，用于对 GitHub 开源项目进行多维探索性分析。系统以 GitHub REST API 和 GitHub Search API 为主要数据来源，采集仓库基础指标、近期提交、Issue、Pull Request、贡献者公开地域信息和 README 等文本语料，并通过离线流水线完成数据清洗、特征工程、机器学习聚类、异常检测和文本语义分析。前端使用 Reflex 构建多页面交互界面，使用 Plotly 绘制统计分布、地域地图、时间趋势、聚类散点图和文本相似度热力图。实验数据包含 200 个真实 GitHub 项目、5,515,757 个 Stars、1,220,277 个 Forks、3,878 条贡献者样本、1,876 条 Issue、6,328 条 Pull Request 以及 40,000 条项目文本相似度记录。结果表明，该系统能够从热度、活跃度、健康度、国际化、时间变化、机器学习结构和文本语义等角度支持开源项目比较与风险识别。本文最后讨论了公开数据偏差、地域字段不确定性、实时榜单依赖网络环境和模型解释粒度等局限，并给出后续改进方向。

关键词：开源软件；GitHub；探索性数据分析；可视化；Reflex；Plotly

目录

1	引言	1
2	数据来源与处理流程	1
2.1	数据来源	1
2.2	离线流水线	2
2.3	数据规模	2
2.4	数学符号与预处理	3
3	系统架构	3
3.1	整体结构	3
3.2	页面组成	4
4	指标体系设计	4
4.1	热度评分	5
4.2	活跃度评分	6
4.3	健康度评分	7
4.4	国际化评分	7
5	机器学习与文本分析方法	7
5.1	建模特征矩阵	8
5.2	标准化	9
5.3	K-Means 聚类	9
5.4	PCA 降维	9
5.5	Isolation Forest 异常检测	10
5.6	聚类评价指标	10
5.7	文本向量、TF-IDF 与余弦相似度	11
5.8	方法与页面对应关系	11
6	可视化设计与系统展示	11
6.1	可视化编码原则	12
6.2	首页总览	13
6.3	项目详情与个体分析	14
6.4	地域可视化	15
6.5	统计分布与相关性	15
6.6	时间趋势	16
6.7	实时榜单	16
6.8	机器学习分析	18
6.9	文本语义分析	19
7	实验结果与分析	19
7.1	总体样本特征	19
7.2	活跃项目观察	20
7.3	聚类画像	20
7.4	模型评估	21
8	系统实现特点	21
8.1	页面渲染与离线任务解耦	21
8.2	缺失数据处理	21
8.3	交互性能优化	22
9	局限性	22
10	结论与展望	22
	附录	24
A	系统页面总览图	24
B	补充具体图表截图	26

1 引言

GitHub 已经成为开源项目协作、分发和社区治理的重要平台。对于使用者、课程研究者和开源维护者而言，项目是否值得关注并不能只通过 Stars 数量判断。一个项目可能拥有很高的历史热度，但近期维护活跃度下降；也可能 Stars 规模较小，却具有较高的提交频率和社区增长潜力；还有一些项目在 Issue 积压、Pull Request 合并率或贡献者地域集中度上表现出维护风险。因此，开源项目分析需要从规模、活跃、健康、协作、地域、文本语义和时间变化等多个维度综合观察。

OpenSourceLens 的目标是把分散的 GitHub 指标转化为可解释、可比较、可交互的分析视图。系统不是静态图表集合，而是一个完整的数据产品原型：首先通过脚本采集或生成数据，再完成清洗、特征工程、机器学习建模和文本分析，最后由 Reflex 前端读取本地处理结果并展示。这样的设计使页面渲染与数据采集、模型训练解耦，避免在用户打开页面时自动调用 GitHub API 或进行高成本训练。

本文的主要工作包括：

1. 构建面向 GitHub 开源项目的离线数据流水线，覆盖采集、清洗、特征构建、模型训练和文本语义分析。
2. 设计热度、活跃度、健康度、国际化和早期增长势能等指标体系，用于支持项目横向比较。
3. 使用 K-Means、PCA 和 Isolation Forest 对项目结构与异常项目进行建模分析。
4. 使用 Reflex 与 Plotly 构建中文多页面交互式可视化系统。

2 数据来源与处理流程

2.1 数据来源

系统的主要数据来源为 GitHub 公开接口和公开网页。仓库元数据、近期活动、贡献者和 README 等信息通过 GitHub REST API 采集，其中仓库搜索依赖 GitHub Search API，仓库详情依赖 Repository 相关 API 端点 [1, 2]。实时榜单页面中，Trending 榜单来自 GitHub Trending 页面 [3]，Stars 实时榜单通过 GitHub Search API 按 Stars 降序查询。

采集范围包括仓库基础信息、近期 Commit、Issue、Pull Request、贡献者样本、贡献者公开 profile 中的 location 字段、README 文本、Topics 和描述字段。需要强调的是，贡献者地域信息来自用户自行填写的公开 location 字段，存在缺失、模糊、不准确或虚构的可能，因此本文将地域分析结果视为近似分布，而不是精确地理测量。

2.2 离线流水线

OpenSourceLens 遵循“数据先于页面”的流程，完整流水线如图 1 所示：



图 1: OpenSourceLens 离线数据处理流水线

从流程关系看，GitHub 原始数据并不是直接进入页面，而是先由离线脚本完成清洗、特征构造、Stars 增长记录、模型训练和文本分析；前端最终读取的是 `data/processed/` 中已经固化的数据产物，因此展示结果具有较好的稳定性和可复现性。

1. 使用 `scripts/collect_github_data.py` 采集真实 GitHub 数据，或使用 `scripts/generate_demo_data.py` 生成演示数据。
2. 使用 `scripts/clean_data.py` 统一字段、补齐缺失列并规范日期和数值类型。
3. 使用 `scripts/build_features.py` 构建项目评分与派生指标。
4. 使用 `scripts/build_star_growth.py` 构建 Stars、Forks 和 Open Issues 历史变化结果。
5. 使用 `scripts/train_models.py` 训练标准化器、K-Means、PCA 和 Isolation Forest。
6. 使用 `scripts/build_text_features.py` 生成文本嵌入、文本聚类 and 项目相似度。
7. Reflex 应用只读取 `data/processed/` 中的 CSV、JSON、NPY 和模型结果文件。

该流程的核心优势是可复现、可检查、可展示。采集和建模都在页面外完成，前端不会在渲染时训练模型，也不会在用户未触发操作时自动请求 GitHub。

2.3 数据规模

本次报告使用的本地处理结果来自 2026 年 5 月 11 日的真实 GitHub API 采集批次。根据 `refresh_metadata.json`，采集项目数为 200，失败项目数为 0。主要数据规模见表 1。

表 1: OpenSourceLens 当前 processed 数据规模

数据文件	数据行数	关键规模	用途
scores.csv	200	5,515,757 Stars	项目评分与基础指标
contributors.csv	3,878	10 个国家	贡献者样本与社区规模
geo_contributors.csv	1,139	12 个城市	地域可视化
issues.csv	1,876	–	Issue 状态与主题
pulls.csv	6,328	–	PR 状态与协作效率
ml_results.csv	200	16 个异常项目	聚类 and 异常检测
text_results.csv	200	6 类主题	文本语义分析
text_similarity.csv	40,000	–	项目相似度矩阵
star_history.csv	17,883	–	时间序列趋势

2.4 数学符号与预处理

为便于后续说明，设共有 n 个开源项目，第 i 个项目的原始观测向量记为

$$\mathbf{x}_i = (x_{i1}, x_{i2}, \dots, x_{ip})^\top, \quad i = 1, 2, \dots, n, \quad (1)$$

其中 p 为参与计算的特征数量。对于 Stars、Forks、Watchers 等典型长尾变量，系统先进行对数压缩：

$$x'_{ij} = \log(1 + x_{ij}), \quad (2)$$

再使用 Min-Max 归一化将其映射到 $[0, 1]$ ：

$$\text{norm}(x_{ij}) = \frac{x_{ij} - \min_i x_{ij}}{\max_i x_{ij} - \min_i x_{ij} + \epsilon}. \quad (3)$$

其中 ϵ 是一个很小的正数，用于避免分母为 0。对 Issue 关闭率、PR 合并率等已经处于 $[0, 1]$ 的比例变量，系统保留其原始比例含义；对缺失字段，清洗脚本会先补齐默认列，再在特征工程阶段按数值字段的合理默认值或空状态处理，避免页面和模型训练因缺列中断。

3 系统架构

3.1 整体结构

系统采用“数据输入层、离线处理层、数据产物层、应用支撑层、交互展示层”五层结构，整体关系如图 2 所示。离线层负责 GitHub 数据采集、清洗、特征工程、机器

学习训练和文本分析；数据产物层以 CSV、JSON、NPY 和 `models/*.pkl` 作为稳定接口；展示层使用 Reflex 构建多页面应用，使用 Plotly 绘制交互式图表 [4, 5]。

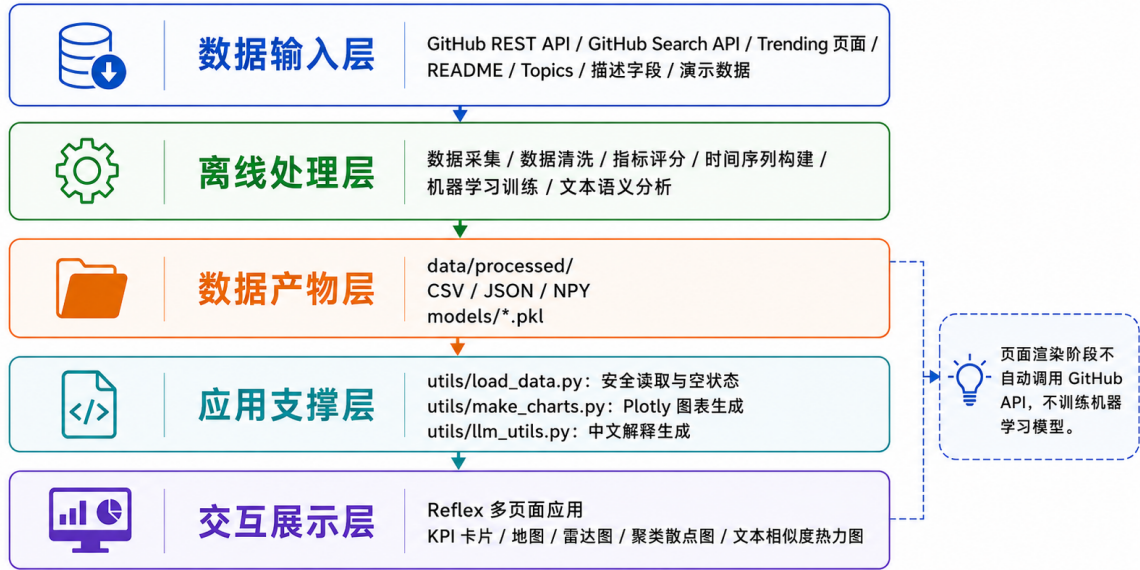


图 2: OpenSourceLens 系统总体结构

这一架构把计算任务和界面渲染清楚分开：脚本层负责产出数据和模型结果，CSV、JSON、NPY 与模型文件作为中接口，Reflex 页面只围绕这些本地文件组织交互和图表展示。

当前项目目录中，数据流水线脚本位于 `scripts/`，前端入口为 `opensource_lens/opensource_lens.py`，页面位于 `opensource_lens/pages/`，可复用组件位于 `opensource_lens/components/`，数据读取和图表生成工具位于 `opensource_lens/utils/`。数据读取集中在 `utils/load_data.py`，图表函数集中在 `utils/make_charts.py`，避免将复杂数据逻辑分散在页面文件中。

3.2 页面组成

系统当前包含 10 个页面，如表 2 所示，覆盖总览、单项目分析、横向对比、地域分布、统计分布、时间趋势、实时榜单、机器学习和文本分析。

4 指标体系设计

指标体系的设计目标不是给项目贴绝对优劣标签，而是把不同量纲的 GitHub 行为统一到可比较的评分空间。设计依据主要来自 CHAOSS 社区的开源项目度量框架，覆盖项目可见度、代码活动、Issue/PR 协作效率、贡献者规模和贡献者地域分布 [6, 7, 8, 9]。已有 GitHub 挖掘研究也指出，Stars 能反映项目关注度但不能单独代表质量 [10]；PR 接纳情况则与协作流程和维护者决策密切相关 [11, 12]。

表 2: OpenSourceLens 前端页面组成

路由	页面	主要功能
/	首页总览	KPI 指标、生态气泡、语言分布、Stars/Forks 排行、首页地域视图
/project	项目详情	单项目雷达图、近期活动、Issue/PR 状态、贡献者国家分布
/compare	项目对比	2 到 5 个项目的评分、雷达图和协作效率对比
/geo	地域可视化	国家地图、城市气泡、国家/城市 Top、项目-国家热力图
/statistics	统计分布与相关性	分布、箱线图、语言分组评分和相关性热力图
/trends	时间序列趋势	Stars、Forks、Open Issues 历史快照变化
/growth	Trending 实时榜	用户点击后抓取 GitHub Trending 页面
/stars	Stars 实时榜	用户点击后调用 GitHub Search API 查询 Stars 排名
/ml	机器学习分析	K-Means、PCA、Isolation Forest、模型评估和聚类画像
/text	文本语义分析	README 嵌入、项目相似度、Issue 主题和 Commit 类型

如图 3 所示, 本文将指标分为热度、活跃度、健康度和国际化四类。设项目 i 的四个核心评分分别为 P_i 、 A_i 、 H_i 和 G_i 。所有评分均限制在 $[0, 1]$, 用于雷达图、热力图、散点图和项目对比。本文权重是结合课程展示目标、字段可获得性和可解释性设置的启发式综合评分, 用于探索性比较而非因果评价。

四类评分在这里共同构成后续分析的坐标系。热度、活跃度、健康度和国际化分别对应不同输入字段, 使用时更适合作为横向比较和探索线索, 而不是直接给项目下质量结论。

4.1 热度评分

热度评分用于衡量项目的外部关注和复用规模。CHAOSS 的 Project Popularity 指标明确将 forks、stars、contributors、downstream dependencies 等作为开源项目可见度和社区采用程度的观测点 [6]。本文在可采集字段范围内使用 Stars、Forks 和 Watchers

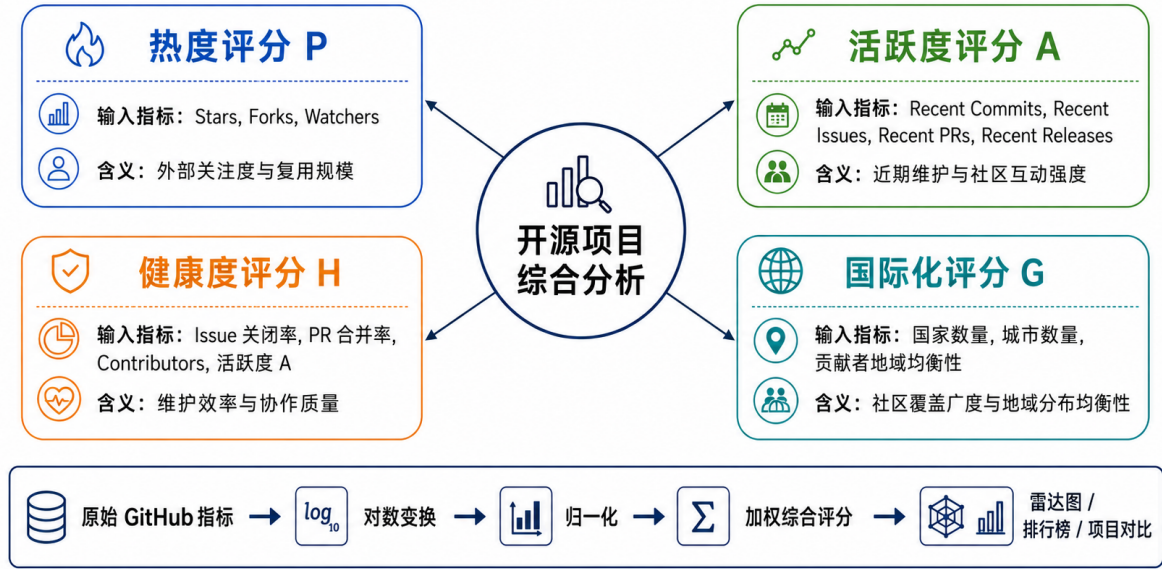


图 3: OpenSourceLens 指标体系设计

三类指标，先进行对数变换以缓解长尾分布，再进行归一化：

$$\begin{aligned}
 P = & 0.5 \cdot \text{norm}(\log(1 + \text{stars})) \\
 & + 0.3 \cdot \text{norm}(\log(1 + \text{forks})) \\
 & + 0.2 \cdot \text{norm}(\log(1 + \text{watchers})).
 \end{aligned} \tag{4}$$

其中 P 表示 `popularity_score`， $\text{norm}(\cdot)$ 表示归一化函数。该评分更适合比较规模差异明显的项目，因为直接使用 Stars 会使头部项目过度支配可视化结果。

4.2 活跃度评分

活跃度评分用于反映项目近期维护和社区互动强度，主要由近期 Commit、Issue 和 Pull Request 构成。CHAOSS 的 Project Velocity 使用 issues、pull requests、commits 和 contributors 衡量项目发展速度，Code Changes Commits 也将提交量视为开发、维护和响应性的观测信号 [7, 13]：

$$\begin{aligned}
 A = & 0.4 \cdot \text{norm}(\text{commits_recent}) \\
 & + 0.3 \cdot \text{norm}(\text{prs_recent}) \\
 & + 0.2 \cdot \text{norm}(\text{issues_recent}) \\
 & + 0.1 \cdot \text{release_recent}.
 \end{aligned} \tag{5}$$

当 release 信息不可用时，脚本会忽略该项并重新归一化权重。当前数据中平均活跃度评分为 0.433。

4.3 健康度评分

健康度评分关注项目维护效率和社区协作质量。Issue 关闭量可反映社区解决问题和完成工作的能力，Change Request Acceptance Ratio 可反映项目对变更请求的接纳、代码评审效率和协作文化 [14, 12]。因此系统综合 Issue 关闭率、PR 合并率、贡献者数量和活跃度：

$$\text{issue_close_rate}_i = \frac{\text{closed_issues}_i}{\text{closed_issues}_i + \text{open_issues_recent}_i + \epsilon}, \quad (6)$$

$$\text{pr_merge_rate}_i = \frac{\text{merged_prs}_i}{\text{closed_prs}_i + \text{open_prs_recent}_i + \epsilon}. \quad (7)$$

$$H = 0.25 \cdot \text{issue_close_rate} + 0.25 \cdot \text{pr_merge_rate} + 0.25 \cdot \text{norm}(\text{contributors}) + 0.25 \cdot A. \quad (8)$$

当前数据中平均健康度评分为 0.441。健康度不是“项目质量”的绝对结论，而是对当前采集窗口内维护状态的近似度量。

4.4 国际化评分

国际化评分用于观察社区贡献者地域分布的广度与均衡性。CHAOSS 的 Contributor Location 指标认为贡献者地域可以反映项目全球触达、工作方式和时区覆盖情况，同时也提醒 profile location 可能存在虚假或无意义填写 [9]。系统使用贡献者国家数量、城市数量和非最大国家占比：

$$\text{top_country_ratio}_i = \frac{\max_c \text{contrib}_{i,c}}{\sum_c \text{contrib}_{i,c} + \epsilon}, \quad (9)$$

其中 $\text{contrib}_{i,c}$ 表示项目 i 在国家或地区 c 的贡献者样本数。 $1 - \text{top_country_ratio}_i$ 越大，说明贡献者越不集中于单一国家或地区。

$$G = 0.4 \cdot \text{norm}(\text{country_count}) + 0.3 \cdot \text{norm}(\text{city_count}) + 0.3 \cdot (1 - \text{top_country_ratio}). \quad (10)$$

由于 GitHub location 字段由用户自行填写， G 只能表示公开样本中的近似国际化程度。

5 机器学习与文本分析方法

机器学习与文本分析流程如图 4 所示。数值特征分支用于聚类、降维和异常检测，文本分支用于构建项目语义向量、主题标签和相似度矩阵，二者共同支撑机器学习分析

页和文本语义分析页。

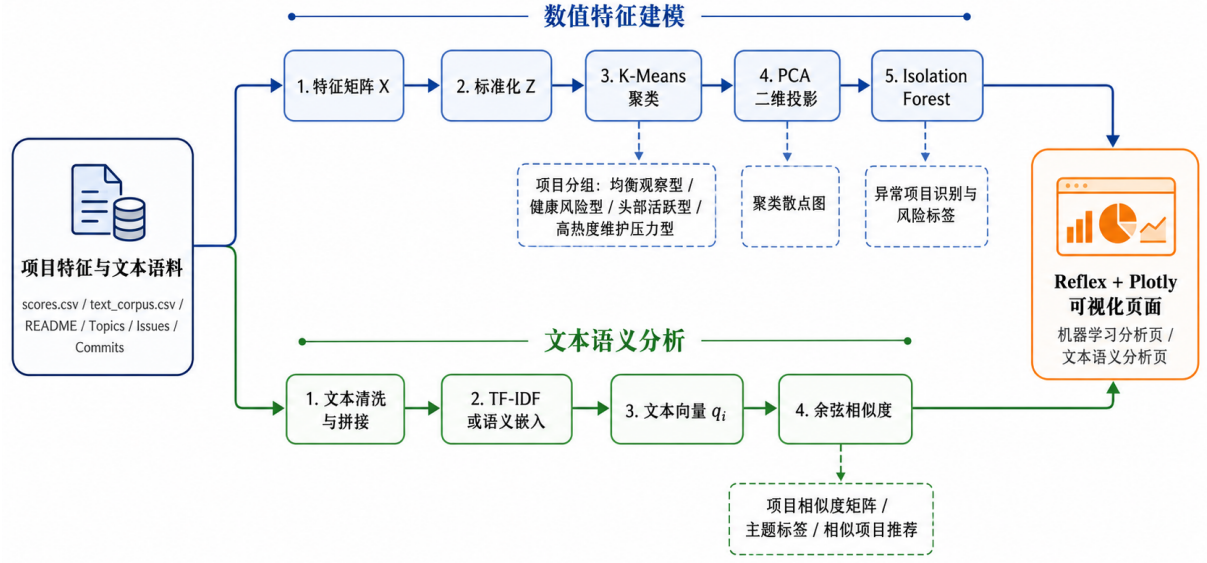


图 4: OpenSourceLens 机器学习与文本分析流程

因此，后续模型实际上沿着两条路径展开：一条从数值指标进入标准化、聚类、降维和异常检测，另一条从 README 和描述文本进入语义嵌入、主题识别与相似度计算，最终共同服务于模型分析页和文本分析页。

5.1 建模特征矩阵

机器学习流水线使用 pandas 读取特征表并进行数值处理 [15]。模型训练由 scikit-learn 实现 [16]。设参与建模的项目数为 n ，数值特征数为 p ，则输入矩阵为

$$X = \begin{bmatrix} \mathbf{x}_1^\top \\ \vdots \\ \mathbf{x}_n^\top \end{bmatrix} \in \mathbb{R}^{n \times p}. \quad (11)$$

本次建模使用 13 个输入特征，主要包括 Stars、Forks、Open Issues、贡献者数、近期 Commit、近期 Issue、近期 PR、Issue 关闭率、PR 合并率、热度评分、活跃度评分、健康度评分和国际化评分。这些特征分别覆盖规模、近期行为、协作效率和社区分布四类信息。

5.2 标准化

由于不同特征量纲差异较大，模型训练前需要进行标准化。对第 j 个特征，StandardScaler 使用均值 μ_j 和标准差 σ_j 计算：

$$z_{ij} = \frac{x_{ij} - \mu_j}{\sigma_j + \epsilon}, \quad \mu_j = \frac{1}{n} \sum_{i=1}^n x_{ij}, \quad \sigma_j = \sqrt{\frac{1}{n} \sum_{i=1}^n (x_{ij} - \mu_j)^2}. \quad (12)$$

标准化后的矩阵记为 $Z \in \mathbb{R}^{n \times p}$ 。这样可以避免 Stars、Forks 等大数值变量在欧氏距离中支配聚类结果，使关闭率、合并率和评分变量也能参与结构识别。

5.3 K-Means 聚类

K-Means 用于把行为相似的项目划分到同一组 [17]。设聚类数为 K ，第 k 个簇的中心为 μ_k ，项目 i 的簇标签为 c_i ，则 K-Means 的优化目标是最小化簇内平方和：

$$\min_{\{c_i\}, \{\mu_k\}} J = \sum_{i=1}^n \|\mathbf{z}_i - \mu_{c_i}\|_2^2. \quad (13)$$

其中 $\mathbf{z}_i$ 是标准化后的项目特征向量。算法迭代执行两个步骤：第一，根据最近的簇中心分配项目标签；第二，根据当前簇内样本重新计算中心：

$$c_i = \arg \min_k \|\mathbf{z}_i - \mu_k\|_2^2, \quad \mu_k = \frac{1}{|C_k|} \sum_{i:c_i=k} \mathbf{z}_i. \quad (14)$$

本次系统设置 $K = 4$ ，得到均衡观察型、健康风险型、头部活跃型和高热度维护压力型四类项目。聚类结果不是监督学习意义上的真实类别，而是帮助发现开源项目行为结构的探索性分组。

5.4 PCA 降维

PCA 用于将高维特征投影到二维平面，便于在前端绘制聚类散点图 [18]。设标准化矩阵为 Z ，其协方差矩阵为

$$S = \frac{1}{n-1} Z^\top Z. \quad (15)$$

PCA 通过特征分解寻找最大方差方向：

$$S \mathbf{v}_j = \lambda_j \mathbf{v}_j, \quad \lambda_1 \geq \lambda_2 \geq \cdots \geq \lambda_p. \quad (16)$$

取前两个特征向量组成投影矩阵 $V_2 = [\mathbf{v}_1, \mathbf{v}_2]$ ，二维坐标为

$$Y = ZV_2, \quad Y \in \mathbb{R}^{n \times 2}. \quad (17)$$

第 j 个主成分的解释率为

$$r_j = \frac{\lambda_j}{\sum_{\ell=1}^p \lambda_\ell}. \quad (18)$$

根据 `model_metrics.csv`, PCA 第一主成分解释率为 0.391, 第二主成分解释率为 0.225, 累计解释率为 0.616。这说明二维投影保留了约 61.6% 的主要方差信息, 适合可视化探索, 但不能完全替代原始高维特征。

5.5 Isolation Forest 异常检测

异常项目识别使用 Isolation Forest。该方法通过随机划分特征空间来隔离样本, 较容易被隔离的样本更可能是异常点; 算法来源和实现接口分别参考 Liu 等人的工作与 scikit-learn 文档 [19, 20]。

对样本 $\mathbf{z}_i$, 设 $h(\mathbf{z}_i)$ 为其在随机隔离树中的路径长度, $E[h(\mathbf{z}_i)]$ 为多棵树的平均路径长度。Isolation Forest 的异常分数可写为:

$$s(\mathbf{z}_i, n) = 2^{-\frac{E[h(\mathbf{z}_i)]}{c(n)}}, \quad (19)$$

其中 $c(n)$ 是样本量为 n 时二叉搜索树不成功搜索路径长度的期望近似:

$$c(n) = 2H(n-1) - \frac{2(n-1)}{n}, \quad H(m) = \sum_{r=1}^m \frac{1}{r}. \quad (20)$$

当 $s(\mathbf{z}_i, n)$ 越接近 1, 样本越容易被隔离, 越可能是异常项目。本次模型检测出 16 个异常项目, 异常比例为 8%。系统进一步结合规则化解释, 将异常原因映射为“高热度低活跃”“维护压力型”“PR 接纳不足”“国际化不足”等中文标签。异常项目不等同于“坏项目”, 而是表示其行为模式与多数样本差异较大, 适合作为后续人工分析对象。

5.6 聚类评价指标

为了衡量聚类结构, 系统输出 K-Means Inertia 和 Silhouette Score。Inertia 即簇内平方和:

$$\text{Inertia} = \sum_{i=1}^n \|\mathbf{z}_i - \boldsymbol{\mu}_{c_i}\|_2^2. \quad (21)$$

轮廓系数同时考虑簇内紧凑度和簇间分离度。设 a_i 为项目 i 到同簇其他样本的平均距离, b_i 为项目 i 到最近其他簇样本的平均距离, 则样本级轮廓系数为:

$$\text{silhouette}(i) = \frac{b_i - a_i}{\max(a_i, b_i)}. \quad (22)$$

整体轮廓系数为所有样本的平均值:

$$\text{Silhouette} = \frac{1}{n} \sum_{i=1}^n \text{silhouette}(i). \quad (23)$$

本次轮廓系数为 0.215, 说明项目之间可能存在一定结构, 但类别边界并不强。这符合开源生态数据的连续分布特征, 因此聚类结果应作为探索性分组而非严格分类结论。

5.7 文本向量、TF-IDF 与余弦相似度

文本分析模块优先读取 `text_corpus.csv`，使用 README、Topics、描述、Issue 标题和 Commit 信息构造项目文本表示。若环境中可用语义嵌入模型，则生成高维文本向量；若缺少相关依赖，则退化为 TF-IDF。

在 TF-IDF 方案中，设词项 t 在项目文档 d 中出现频率为 $\text{tf}(t, d)$ ，包含词项 t 的文档数为 $\text{df}(t)$ ，总文档数为 N ，则逆文档频率为：

$$\text{idf}(t) = \log \frac{N + 1}{\text{df}(t) + 1} + 1. \quad (24)$$

词项权重为：

$$w_{t,d} = \text{tf}(t, d) \cdot \text{idf}(t). \quad (25)$$

项目文本向量记为 $\mathbf{q}_i$ ，两个项目之间的文本相似度使用余弦相似度：

$$\text{sim}(i, j) = \frac{\mathbf{q}_i^\top \mathbf{q}_j}{\|\mathbf{q}_i\|_2 \|\mathbf{q}_j\|_2 + \epsilon}. \quad (26)$$

当前系统已经生成三类文本分析产物：

- `text_embeddings.npy`：保存项目文本向量；
- `text_results.csv`：保存二维坐标、主题标签与主题关键词；
- `text_similarity.csv`：保存项目之间的文本相似度矩阵。

文本页面中的项目相似度热力图优先使用高维向量的余弦相似度，而不是仅凭二维嵌入坐标距离判断相似性。

5.8 方法与页面对应关系

表 3 汇总了本文使用的主要数学方法、输入数据和页面用途。可以看到，OpenSourceLens 的每个页面都对应明确的数据变换或统计建模逻辑，而不是仅对原始字段进行静态展示。

6 可视化设计与系统展示

本节配图来自项目网页实际截图。这些图像保存在 `report/figures/` 中，用于展示当前系统界面和分析结果。

表 3: 主要数学方法与系统页面对应关系

方法	输入字段	支撑页面
对数变换与归一化	Stars、Forks、Watchers、近期活动量	首页、项目详情、项目对比
加权综合评分	热度、活跃、健康、国际化相关字段	首页、对比页、雷达图、排行榜
Pearson 相关系数	核心数值指标矩阵	统计分布与相关性页面
累计变化量	Stars/Forks/Open Issues 历史快照	时间序列趋势页面
K-Means 聚类	13 维标准化项目特征	机器学习分析页面
PCA 二维投影	标准化后的高维特征矩阵	聚类散点图、异常散点图
Isolation Forest	标准化后的高维特征矩阵	异常检测、风险项目排行
TF-IDF/文本嵌入	README、Topics、项目描述、Issue 与 Commit 文本	文本语义分析页面
余弦相似度	项目文本向量	项目相似度热力图

6.1 可视化编码原则

系统中的图表并不是简单展示原始表格，而是将指标映射到位置、颜色、面积、形状和排序等视觉通道。图 5 以生态散点图为例，展示了活跃度、健康度、项目规模和国际化程度的主要编码关系：

$$(x_i, y_i, \text{size}_i, \text{color}_i) = (A_i, H_i, \log(1 + \text{stars}_i), G_i), \quad (27)$$

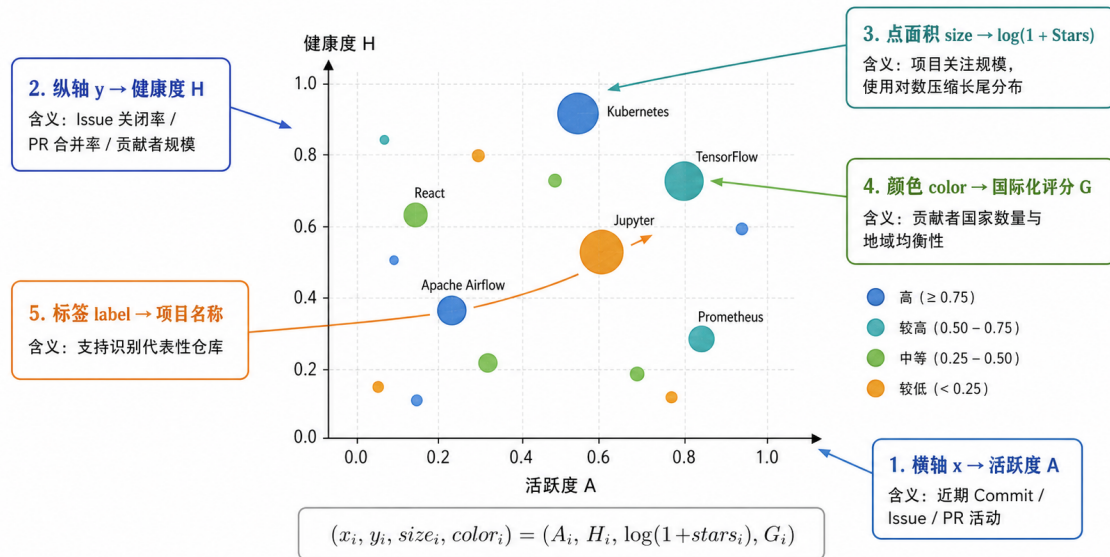


图 5: OpenSourceLens 可视化编码映射

在生态散点图中，位置、面积和颜色被同时利用起来：活跃度和健康度决定项目所在位置，Stars 规模影响点的大小，国际化程度通过颜色呈现，从而把多个指标压缩到同一张探索图中。

这种映射使单张图同时承载多个维度，适合用于探索性发现。

相关性热力图使用 Pearson 相关系数。设两个指标向量为 $\mathbf{u}$ 和 $\mathbf{v}$ ，均值分别为 $\bar{u}$ 和 $\bar{v}$ ，则相关系数为

$$\rho_{uv} = \frac{\sum_{i=1}^n (u_i - \bar{u})(v_i - \bar{v})}{\sqrt{\sum_{i=1}^n (u_i - \bar{u})^2} \sqrt{\sum_{i=1}^n (v_i - \bar{v})^2} + \epsilon}. \quad (28)$$

ρ_{uv} 越接近 1 表示两个指标越倾向于同向变化，越接近 -1 表示越倾向于反向变化，接近 0 则表示线性相关较弱。

时间序列页使用相对首个快照的累计变化量。设项目 i 在第 t 次快照的 Stars 为 $s_{i,t}$ ，则累计变化为

$$\Delta s_{i,t} = s_{i,t} - s_{i,1}. \quad (29)$$

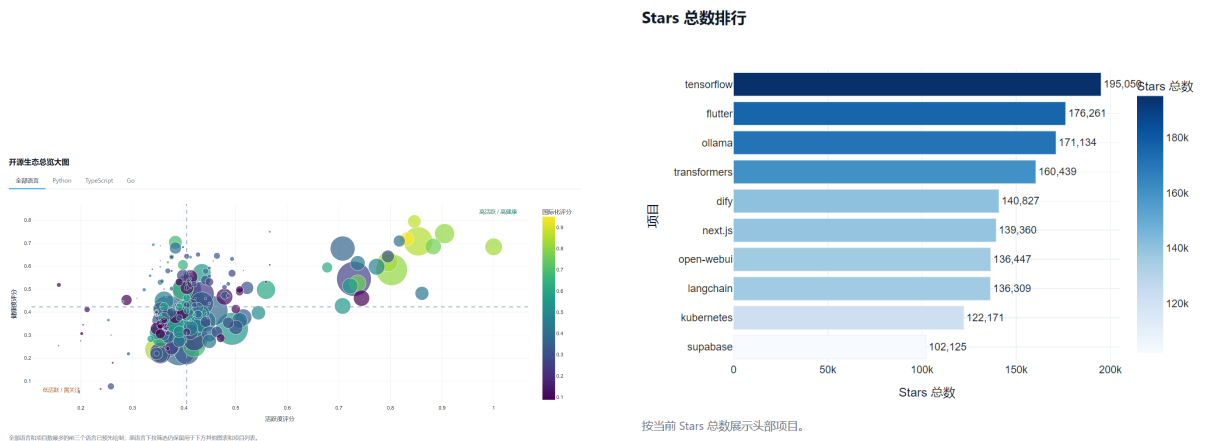
若需要比较不同项目的相对增长，也可以使用归一化变化率：

$$g_{i,t} = \frac{s_{i,t} - s_{i,1}}{s_{i,1} + 1}. \quad (30)$$

当前系统优先展示累计变化量，因为它既能过滤完全不变化的项目，又能避免绝对 Stars 规模过大导致图形不可读。

6.2 首页总览

首页承担全局入口作用，展示项目总数、Stars、Forks、贡献者、覆盖国家、平均健康度、平均活跃度和数据更新时间。整页入口布局见附录图 16a；正文优先展示更具体的分析图表。图 6 给出了首页最核心的两个观察入口：生态散点图用于发现头部项目、潜力项目和风险项目，Stars 排行图用于直接观察关注度最高的项目。



(a) 生态散点图，展示项目在活跃度和健康度空间中的分布。

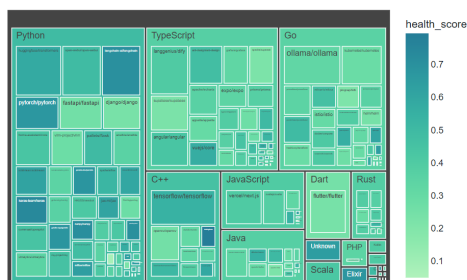
(b) Stars Top 10，展示样本中关注度最高的一组项目。

图 6：首页核心图表：开源生态散点与 Stars Top 10

首页没有只停留在页面外观展示，而是先给出最能支撑分析的两个入口：生态散点图帮助判断项目在多指标空间中的位置，Stars 排行则快速定位样本中的高关注仓库。

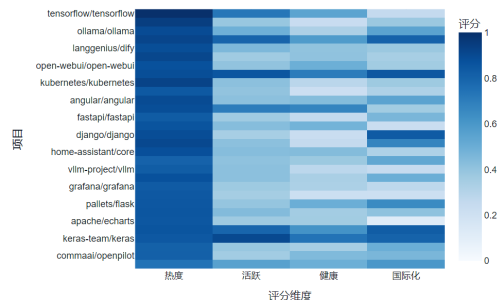
首页还包含语言生态树图、评分热力图、语言分布和 Stars-Forks 关系等图表，如图 7 所示。这些图表用于从不同维度观察开源生态的长尾结构和指标相关性。

语言生态树图



用面积表示不同语言和项目的 Stars 规模，用颜色观察健康度差异。

头部项目评分热力图

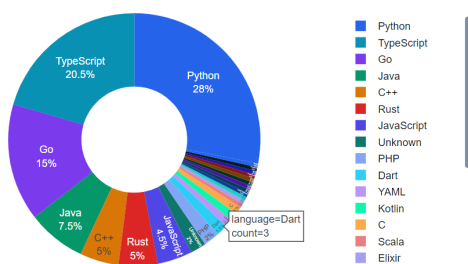


比较头部项目在热度、活跃度、健康度、国际化上的结构差异。

(a) 语言生态树图，展示不同语言项目的规模占比和层级关系。

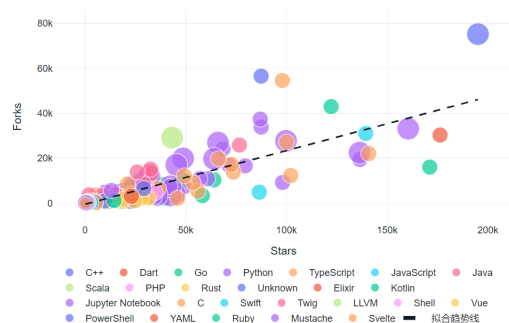
(b) 项目评分热力图，用颜色对比各项目在核心评分上的差异。

语言分布



展示样本项目的主语言结构。

Stars-Forks 关系



观察关注度和复用规模的关系，气泡大小代表贡献者数量。

(c) 语言分布环图，展示样本项目的主要编程语言构成。

(d) Stars-Forks 散点图，观察项目关注度与复用规模之间的关系。

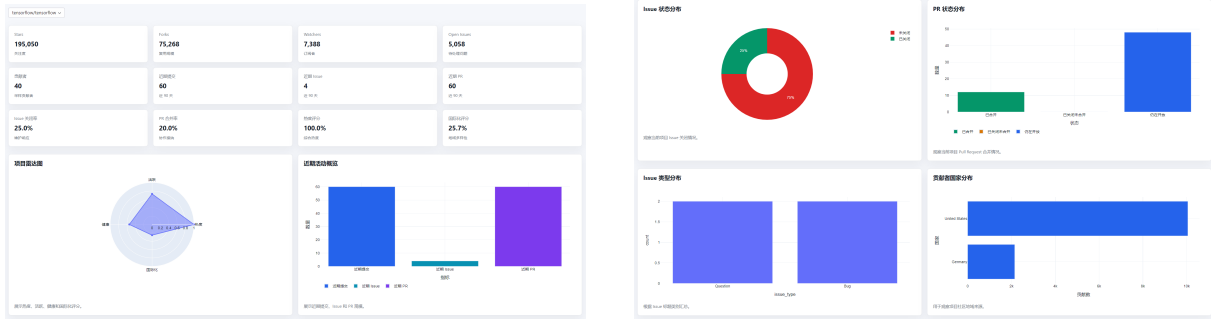
图 7：首页扩展图表：语言生态、评分热力图、语言分布和 Stars-Forks 关系

这些扩展图表进一步把首页观察拆开：树图和环图揭示语言生态结构，评分热力图突出项目间的指标差异，Stars-Forks 散点图则便于发现高关注、高复用以及偏离总体趋势的仓库。

6.3 项目详情与个体分析

项目详情页用于从整体下钻到单个项目，整页布局见附录图 16b。正文重点展示图 8 中的单项目画像关键图表：雷达图能够同时呈现热度、活跃度、健康度和国际化评分，Issue/PR 状态图则用于判断维护压力和协作效率。

进入单项目页面后，分析重点从总体分布转为个体画像。雷达图用来概括该项目的评分长短板，Issue/PR 状态图则把这些评分差异进一步落到维护压力和协作过程上。



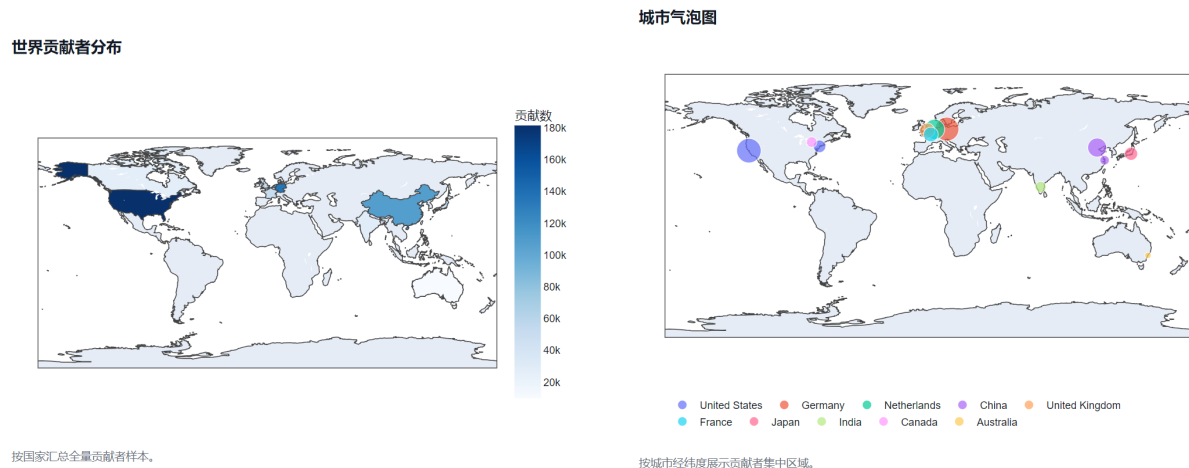
(a) 项目雷达图与近期活动，用于判断单项目优势和短板。

(b) Issue/PR 状态图，用于观察协作效率和维护压力。

图 8：项目详情页具体图表：项目画像与协作状态

6.4 地域可视化

地域页展示贡献者国家分布、城市气泡、国家 Top、城市 Top 和项目-国家热力图，整页总览见附录图 17a。正文选择图 9 中的国家地图与城市气泡图作为代表，因为二者分别对应国家和城市两个空间粒度。由于 location 字段的自填性质，该页面更适合解释为“公开样本中的地域近似分布”，不应被解读为贡献者真实位置的精确统计。



(a) 国家贡献者分布图，用颜色深浅表示公开样本规模。

(b) 城市气泡图，用位置和气泡大小表示城市层面的近似分布。

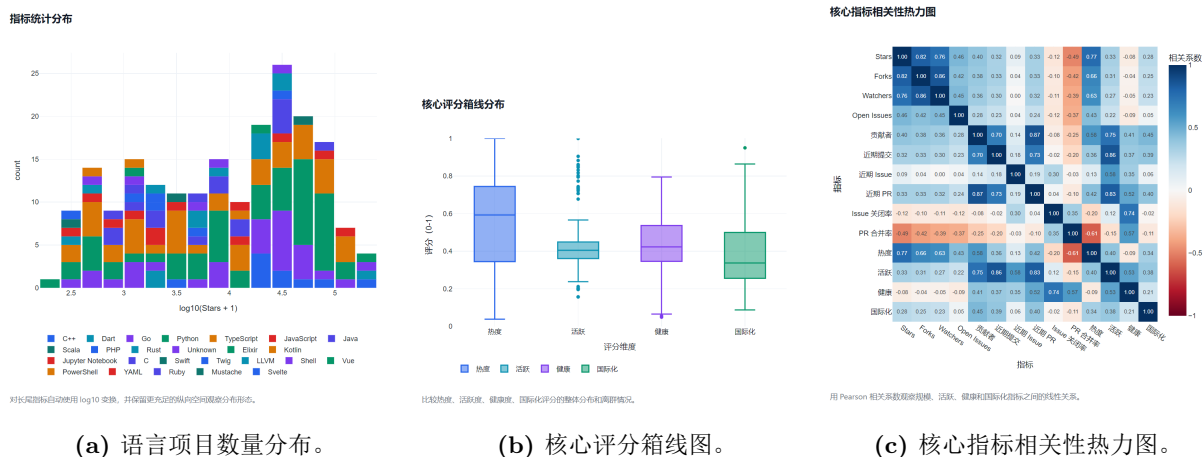
图 9：地域可视化具体图表：国家分布与城市气泡

地域分析选择国家地图和城市气泡两种粒度配合展示：前者提供宏观国家分布，后者补充城市层面的局部线索，二者都应结合 location 字段自填这一前提谨慎解读。

6.5 统计分布与相关性

统计页面用于回答“数据整体长什么样”，整页分布视图见附录图 17b。正文保留图 10 中更适合直接解释的统计图：语言数量分布、核心评分箱线图和指标相关性热力图。箱线图有助于比较不同评分的离散程度；相关性热力图则用于观察规模、活跃、健

康和国际化之间是否存在共变关系。



(a) 语言项目数量分布。

(b) 核心评分箱线图。

(c) 核心指标相关性热力图。

图 10: 统计分布与相关性具体图表

这组三个统计图分别对应样本数量结构、评分离散程度和指标相关关系：语言分布反映技术栈覆盖，箱线图比较核心评分的分散情况，热力图则用于观察不同指标是否存在同向或反向变化。

6.6 时间趋势

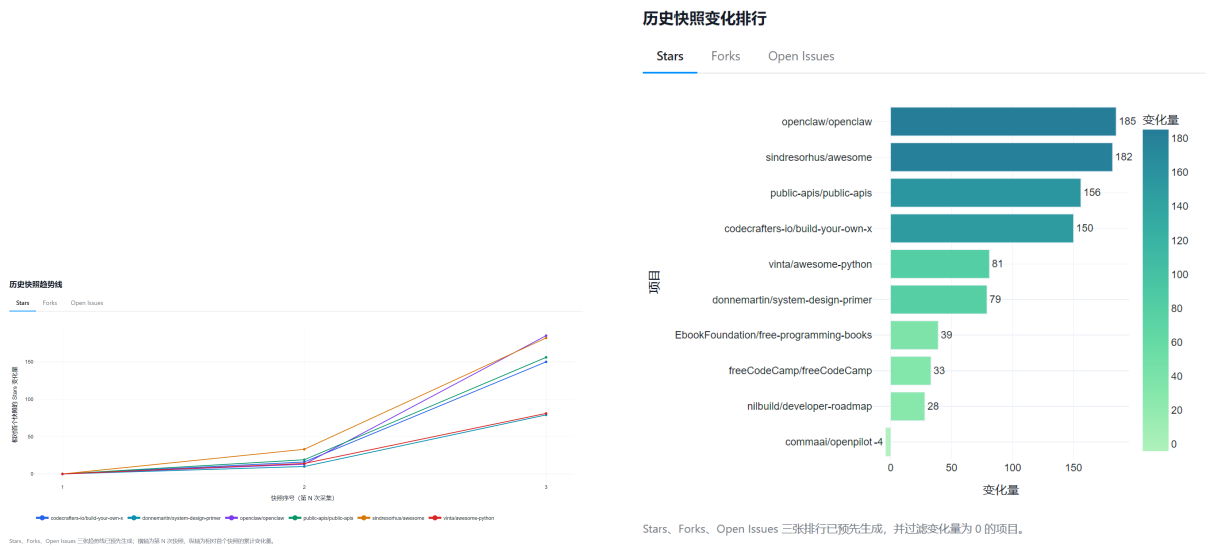
时间序列页读取 `star_history.csv`，展示 Stars、Forks 和 Open Issues 的历史快照变化，整页总览见附录图 18a。正文通过图 11 重点展示 Stars 累计变化折线和增长排行。趋势线使用“相对首个快照的累计变化量”，而不是直接使用绝对 Stars。这样可以减弱头部项目规模过大的影响，使中小项目的变化更容易被观察。

6.7 实时榜单

系统包含两个实时观察页面。Trending 实时榜只在用户点击“实时刷新 Trending”时请求 GitHub Trending 页面，整页入口见附录图 18b，具体结果见图 12；Stars 实时榜只在用户点击查询按钮时调用 GitHub Search API，整页入口见附录图 19a，具体结果见图 13。二者都不会在页面渲染时自动联网，因此不会破坏离线分析的稳定性。

时间趋势在图 11 中被拆成两个互补视角：折线图强调项目随快照变化的过程，增长排行则直接给出同一时间窗口内增长最明显的项目。图 12 展示实时 Trending 结果，一方面用排行图显示用户刷新时 GitHub 上更受关注的仓库，另一方面用语言分布判断热点集中在哪些技术栈。

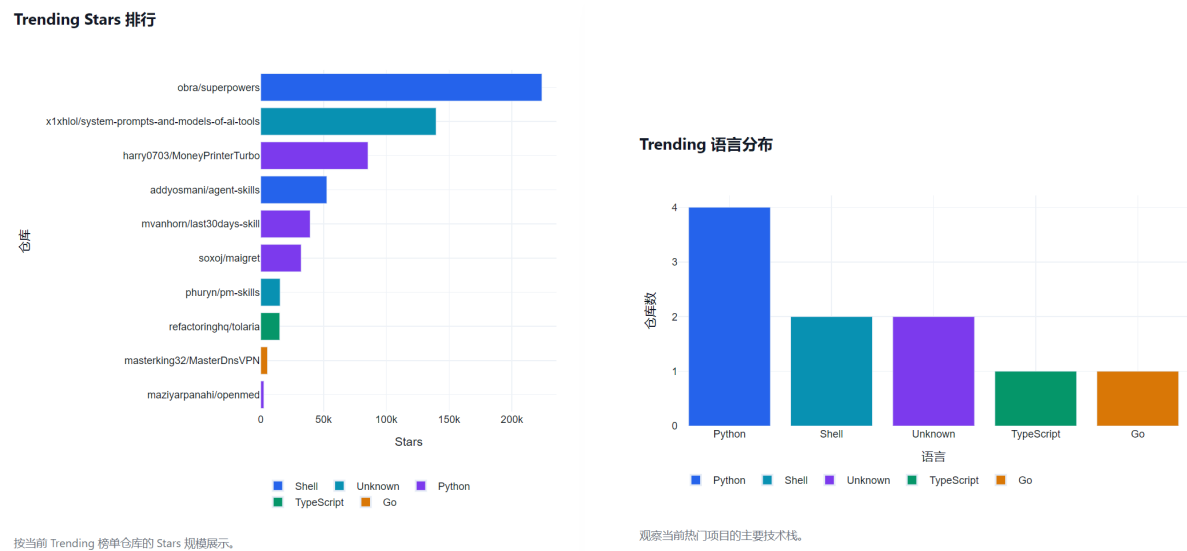
Stars 查询结果则更偏向高关注项目的横截面观察：排行图给出仓库关注度顺序，语言分布帮助判断高 Stars 仓库是否集中在少数编程语言或技术方向上。



(a) Stars 历史趋势折线图。

(b) Stars 增长排行图。

图 11: 时间趋势具体图表: Stars 累计变化与增长排行

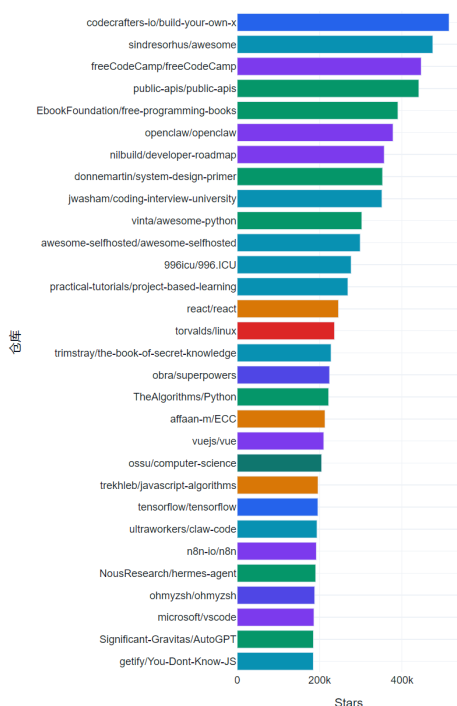


(a) Trending 项目排行图。

(b) Trending 语言分布图。

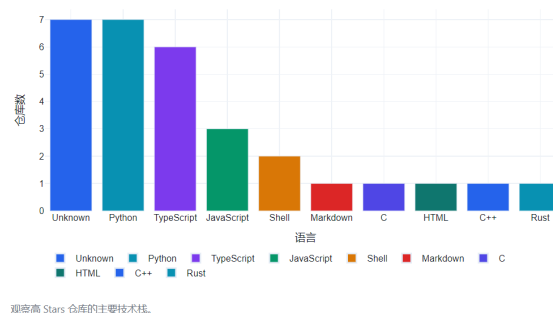
图 12: GitHub Trending 实时榜具体图表

Stars 排行



(a) Stars Top 30 排行图。

语言分布



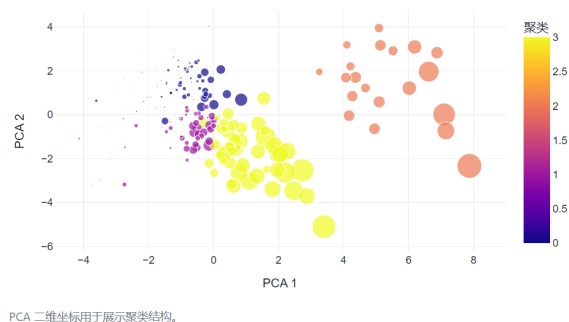
(b) Stars 榜语言分布图。

图 13: Stars 实时榜具体图表

6.8 机器学习分析

机器学习页展示 K-Means 聚类、PCA 二维投影、Cluster 雷达图、Isolation Forest 异常检测、聚类平均指标热力图和风险项目排行，整页总览见附录图 19b。图 14 展示了聚类散点图和异常检测图。红色异常点代表与多数项目行为差异较大的项目，适合后续人工分析。

K-Means 聚类散点图



(a) K-Means 聚类散点图，展示 PCA 降维后的项目分组结构。

Isolation Forest 异常检测



(b) Isolation Forest 异常检测图，红色点表示行为模式差异较大的项目。

图 14: 机器学习分析：K-Means 聚类、PCA 投影和 Isolation Forest 异常检测

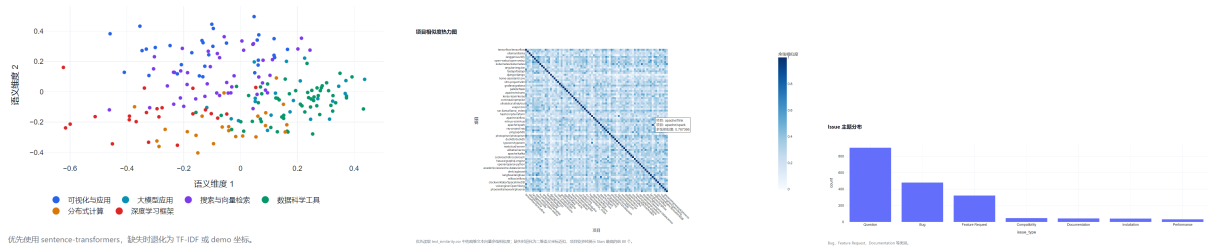
机器学习页通过两类图表把模型结果可视化：聚类散点图展示项目在降维空间中的

群体结构，异常检测图则标出与多数项目行为差异较大的对象，作为后续人工解释的入口。

6.9 文本语义分析

文本语义页展示 README/描述语义嵌入、项目相似度热力图、Commit 类型分布和 Issue 主题分布，整页总览见附录图 20a。图 15 同时展示语义嵌入、相似度热力图和 Issue 主题分布。由于相似度优先来自高维文本向量的余弦相似度，该图比单纯使用二维坐标距离更适合作为语义近似关系的依据。

README 语义嵌入散点图



(a) README 语义嵌入散点图。

(b) 项目文本相似度热力图。

(c) Issue 主题分布图。

图 15: 文本语义分析具体图表

文本分析部分把非结构化信息拆成语义位置、项目相似性和社区反馈主题三层。嵌入散点图提供大致语义分布，相似度热力图呈现项目之间的文本接近关系，Issue 主题图则补充开发者和用户反馈内容。

7 实验结果与分析

7.1 总体样本特征

当前样本包含 200 个 GitHub 项目，总 Stars 为 5,515,757，总 Forks 为 1,220,277，总贡献者样本为 3,878。覆盖公开 location 解析后的 10 个国家和 12 个城市。平均健康度评分为 0.441，平均活跃度评分为 0.433，说明样本中既包含头部成熟项目，也包含活跃度、协作效率和地域覆盖差异较大的中小项目。

表 4 展示 Stars 排名前 10 的项目。可以看到，TensorFlow、Flutter、Ollama、Transformers 等项目在热度上占据明显优势，但它们的活跃度、健康度和国际化评分并不完全一致。这说明热度指标无法单独替代多维评价。

表 4: Stars 排名前 10 项目及核心评分

项目	Stars	Forks	活跃度	健康度
tensorflow/tensorflow	195,050	75,268	0.729	0.545
flutter/flutter	176,261	30,345	0.391	0.239
ollama/ollama	171,134	16,054	0.493	0.327
huggingface/transformers	160,439	33,151	0.802	0.585
langgenius/dify	140,827	22,101	0.456	0.406
vercel/next.js	139,360	31,041	0.369	0.409
open-webui/open-webui	136,447	19,432	0.405	0.497
langchain-ai/langchain	136,309	22,529	0.854	0.708
kubernetes/kubernetes	122,171	43,017	0.420	0.288
supabase/supabase	102,125	12,348	0.405	0.226

7.2 活跃项目观察

按活跃度评分排序，pandas、Keras、Gradio、openai-python、LangChain、XGBoost、NumPy 等项目位于前列。该结果说明传统基础库和新兴 AI 工具链都具有较高近期活动。特别是 LangChain 同时具有较高 Stars 和高活跃度，而 openai-python 的 Stars 规模相对较小但活动水平较高，适合作为“潜力增长型”项目观察对象。

7.3 聚类画像

K-Means 将 200 个项目划分为 4 类。表 5 展示了聚类画像。Cluster 2 的平均热度、活跃度、健康度和国际化评分均较高，可解释为“头部活跃型”；Cluster 3 的平均热度较高但健康度和活跃度相对较低，可解释为“高热度维护压力型”；Cluster 1 的健康度较低，适合作为维护风险观察类别。

表 5: K-Means 聚类画像

聚类	项目数	热度均值	活跃均值	健康均值	解释标签
Cluster 0	64	0.384	0.414	0.562	均衡观察型
Cluster 1	64	0.435	0.348	0.341	健康风险型
Cluster 2	20	0.786	0.796	0.616	头部活跃型
Cluster 3	52	0.789	0.420	0.347	高热度维护压力型

7.4 模型评估

表 6 展示了主要模型指标。K-Means 轮廓系数为 0.215，说明项目之间存在一定结构，但聚类边界并不非常清晰。这符合开源项目数据的实际特征：不同项目在热度、活跃、健康和国际化之间往往连续变化，而不是天然分成完全分离的类别。因此聚类结果更适合作为探索性分组，而不是严格分类标签。

表 6: 机器学习模型指标

指标	数值
样本项目数	200
输入特征数	13
K-Means 聚类数	4
K-Means Inertia	1276.092
Silhouette Score	0.215
PCA 第一主成分解释率	0.391
PCA 第二主成分解释率	0.225
PCA 累计解释率	0.616
异常项目数	16
异常项目占比	0.080

8 系统实现特点

8.1 页面渲染与离线任务解耦

OpenSourceLens 的关键工程原则是避免把重型任务放入页面渲染流程。GitHub 采集、特征工程、模型训练和文本语义分析都通过脚本执行，页面仅读取本地结果。这种设计有三点好处：第一，课程展示时稳定，不依赖实时网络；第二，用户切换页面时不会触发耗时训练；第三，数据和模型产物可以被单独检查，便于定位错误。

8.2 缺失数据处理

前端数据读取函数采用安全读取策略。如果某个 processed 文件不存在，读取函数返回空 DataFrame，页面显示中文空状态，而不是直接抛出异常。这对课程项目尤其重要，因为不同环境可能只生成 Demo 数据或缺少文本嵌入依赖。

8.3 交互性能优化

系统使用两类优化策略。第一类是前端 Tab 预绘制，例如首页语言快速视图、统计页评分指标切换和趋势页指标切换，减少 Reflex 后端状态往返。第二类是进程内图表缓存，项目详情和地域页会基于数据指纹、项目选择和筛选条件缓存 Plotly Figure。数据刷新后，数据指纹变化会使旧缓存失效。

9 局限性

尽管 OpenSourceLens 已经覆盖较完整的探索性分析流程，但仍存在以下局限：

1. **采集时刻依赖。** GitHub 数据会持续变化，本文结果反映的是 2026 年 5 月 11 日采集窗口内的状态，不代表长期稳定结论。
2. **API 限制。** GitHub API 受网络环境、Token 权限和频率限制影响。匿名请求在实时榜单场景中更容易触发限流。
3. **地域字段不确定。** 贡献者 location 字段由用户自填，不保证标准化、完整性和真实性。
4. **Stars 增长估算有限。** 发布一年后 Stars 指标基于当前 Stars 和项目年龄估算，不是真实逐日历史数据。
5. **模型解释粒度有限。** K-Means 和 Isolation Forest 适合发现结构和异常，但异常原因仍主要依赖规则模板，需要更细粒度的特征贡献解释。
6. **数据规模扩展问题。** 当前 200 项目规模适合课程展示；若扩大到数万项目，需要 Parquet、DuckDB 或数据库，并考虑离线图表预聚合。

10 结论与展望

本文设计并实现了 OpenSourceLens，一个基于 GitHub 开源项目数据的中文交互式探索性分析平台。系统从公开 GitHub 数据出发，构建了采集、清洗、特征工程、机器学习、文本分析和 Reflex 可视化的完整流程。当前样本覆盖 200 个真实项目，能够从热度、活跃度、健康度、国际化、时间趋势、机器学习结构和文本语义等多个维度支持开源项目分析。

实验和界面展示表明，多维指标体系能够弥补单一 Stars 指标的不足。机器学习聚类帮助识别头部活跃型、健康风险型和高热度维护压力型项目；异常检测帮助定位行为模式与多数项目不同的仓库；文本语义分析则从 README、描述和主题等非结构化信

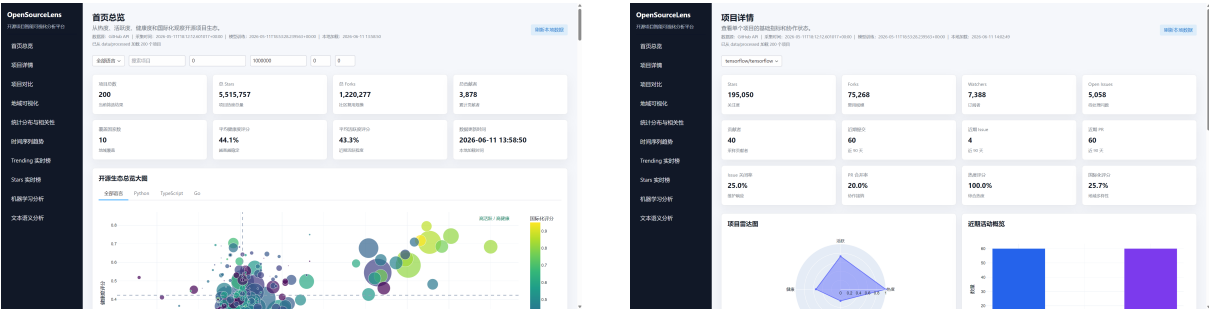
息中补充项目相似性视角。系统的离线流水线设计保证了课程展示的稳定性和可扩展性，也为后续扩展真实更新任务、报告导出和更大规模数据分析提供了基础。

未来工作可以从四个方向推进：第一，增加图表导出和自动课程报告生成功能；第二，引入更精细的 Issue/PR 时间序列和维护者响应时间指标；第三，为异常检测增加特征贡献解释和多随机种子稳定性评估；第四，将重型图表的预聚合结果写入本地缓存或使用 DuckDB/Parquet 支撑更大规模数据。

附录

A 系统页面总览图

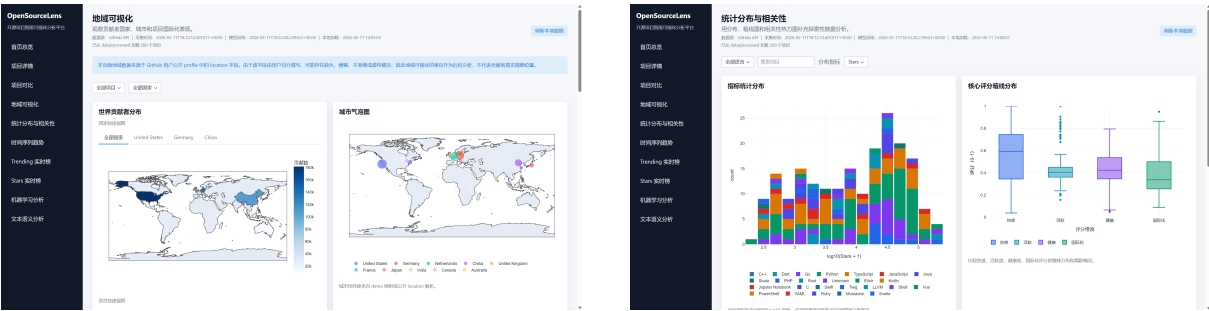
本附录集中放置整页总览截图。正文部分只引用这些总览图来说明页面入口与布局，具体分析过程则优先使用局部图表截图。



(a) 首页总览：KPI 指标、筛选器与生态散点主图。 (b) 项目详情页总览：指标卡片、雷达图与活动概览。

图 16: 首页与项目详情页总览截图

这些总览截图主要承担定位作用。读者可以先通过它们了解首页和详情页的整体布局，再回到正文中查看被单独展开的核心图表。



(a) 地域可视化页总览：国家地图、城市气泡与筛选区域。 (b) 统计分布页总览：语言分布与评分分布面板。

图 17: 地域页与统计页总览截图

地域页和统计页分别代表空间分析与样本分布分析两类入口。前者围绕国家、城市和筛选结果展开，后者则集中呈现语言分布、评分分布和相关性。

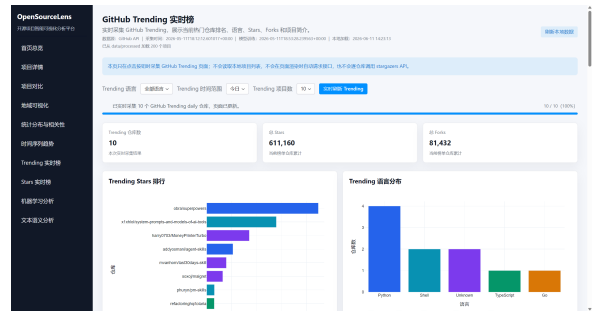
这一组总览图用于交代时间相关页面的界面形态。正文没有重复展开整页截图，而是把重点放在趋势折线、增长排行和实时语言分布等更能说明问题的局部图表上。

Stars 实时榜和机器学习页在交互逻辑上有所不同：前者围绕查询条件和实时排行组织，后者围绕聚类、异常检测及模型解释组织，正文中的具体模型图表与这里的页面布局相互对应。

文本语义页和项目对比页展示了两个专题分析场景的完整结构：一个偏向文本相似性和主题理解，一个偏向多项目评分和协作指标的横向比较。

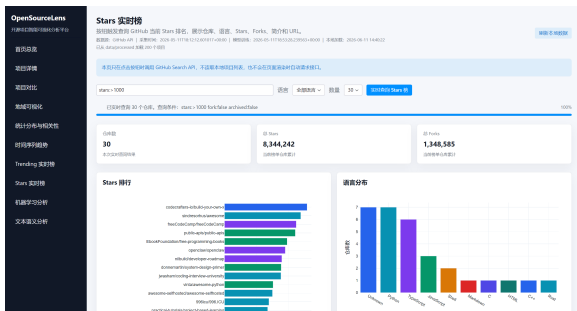


(a) 时间趋势页总览：历史快照折线与增长排行。



(b) Trending 实时榜页总览：刷新控件、KPI 与实时排行。

图 18: 时间趋势页与 Trending 实时榜页总览截图



(a) Stars 实时榜页总览：查询条件、实时指标与榜单图表。



(b) 机器学习页总览：聚类散点图与聚类雷达图。

图 19: Stars 实时榜页与机器学习页总览截图



(a) 文本语义页总览：文本嵌入、主题分布与相似度热力图。



(b) 项目对比页总览：多项目评分与雷达图对比。

图 20: 文本语义页与项目对比页总览截图

B 补充具体图表截图

本附录继续保留未在正文中逐一展开的具体图表截图，作为系统功能覆盖面的补充证据。

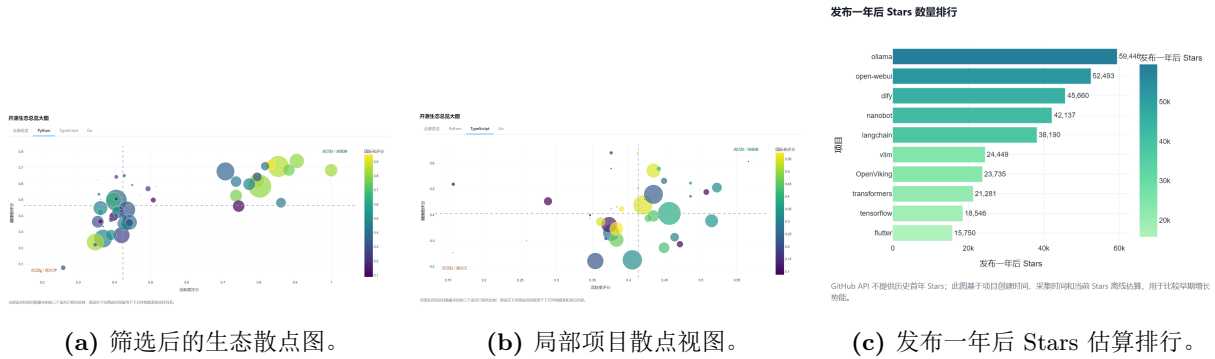


图 21: 首页补充图表截图

这组补充图表体现首页的进一步下钻能力：筛选视图支持从全局散点转向局部项目观察，发布一年后 Stars 估算则补充了增长潜力这一时间维度。

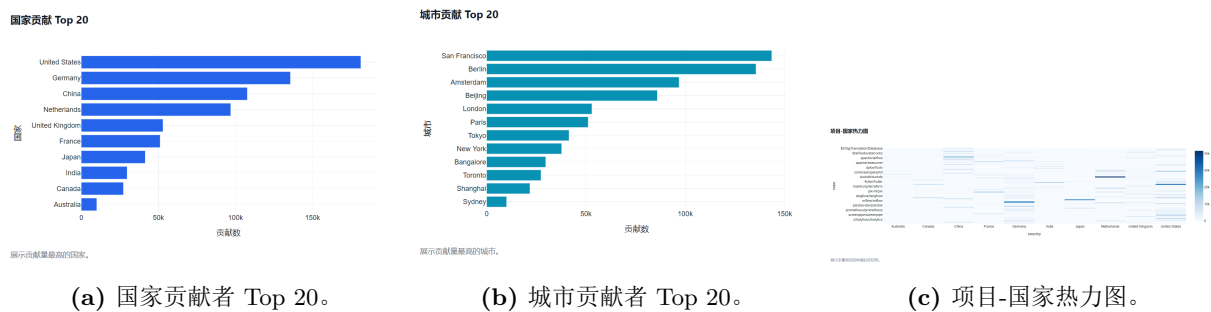


图 22: 地域分析补充图表截图一

地图之外，国家 Top、城市 Top 和项目-国家热力图把空间信息转化为排序和交叉关系，便于比较不同国家、城市以及项目之间的地域覆盖差异。

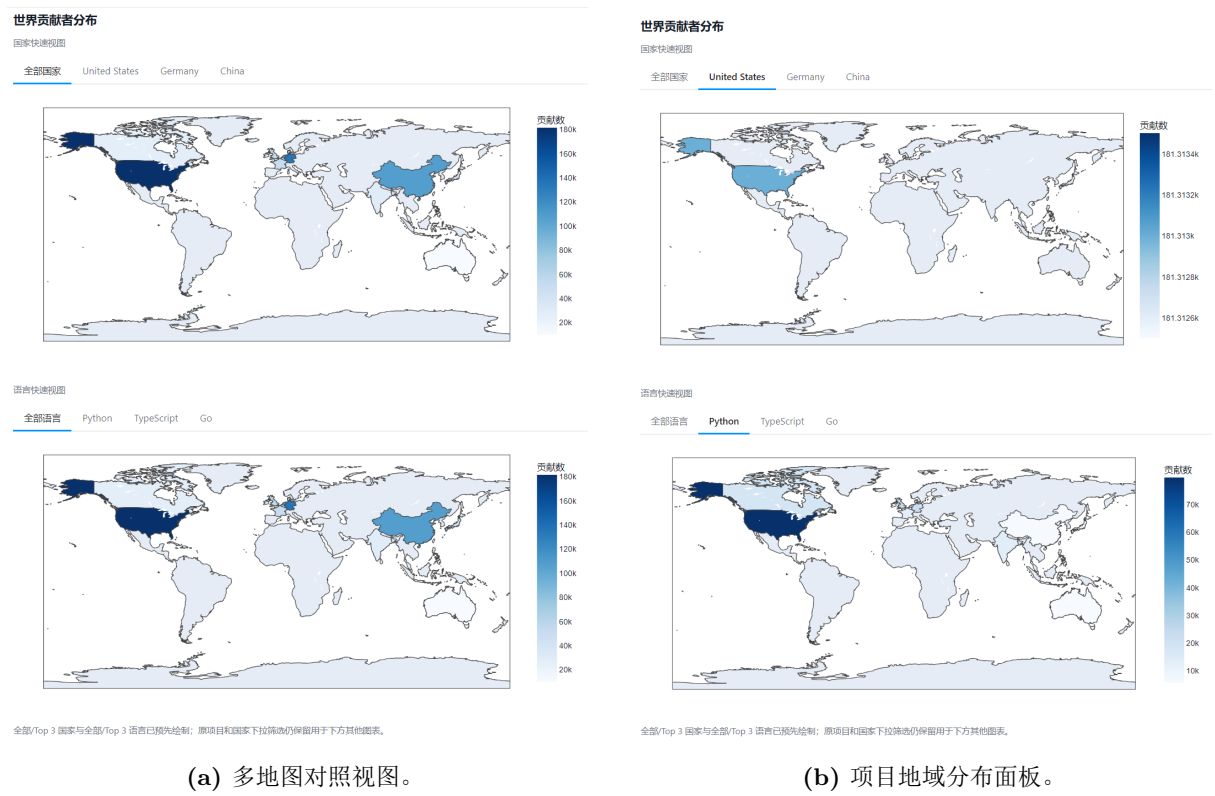
多地图和项目面板共同展示了地域页的多视图验证方式。对于同一批 location 数据，可以从地图形态和项目维度两个角度交叉检查；国家与城市排行已经在前一组图中分别展示，因此这里不再重复放置拼接图。

交互筛选进一步把地域分析从全局概览推进到局部观察。用户可以围绕某个项目或某个国家重新组织地图结果，从而定位更具体的贡献者分布线索。

按语言分组后，活跃度和热度箱线图可以比较不同技术栈在近期维护强度和外部关注度上的差异，帮助判断语言生态之间是否存在明显分层。

健康度和国际化箱线图则补充了协作效率与地域覆盖视角，使统计页不只比较项目规模，也能够观察不同语言社区在维护质量和贡献者分布上的差异。

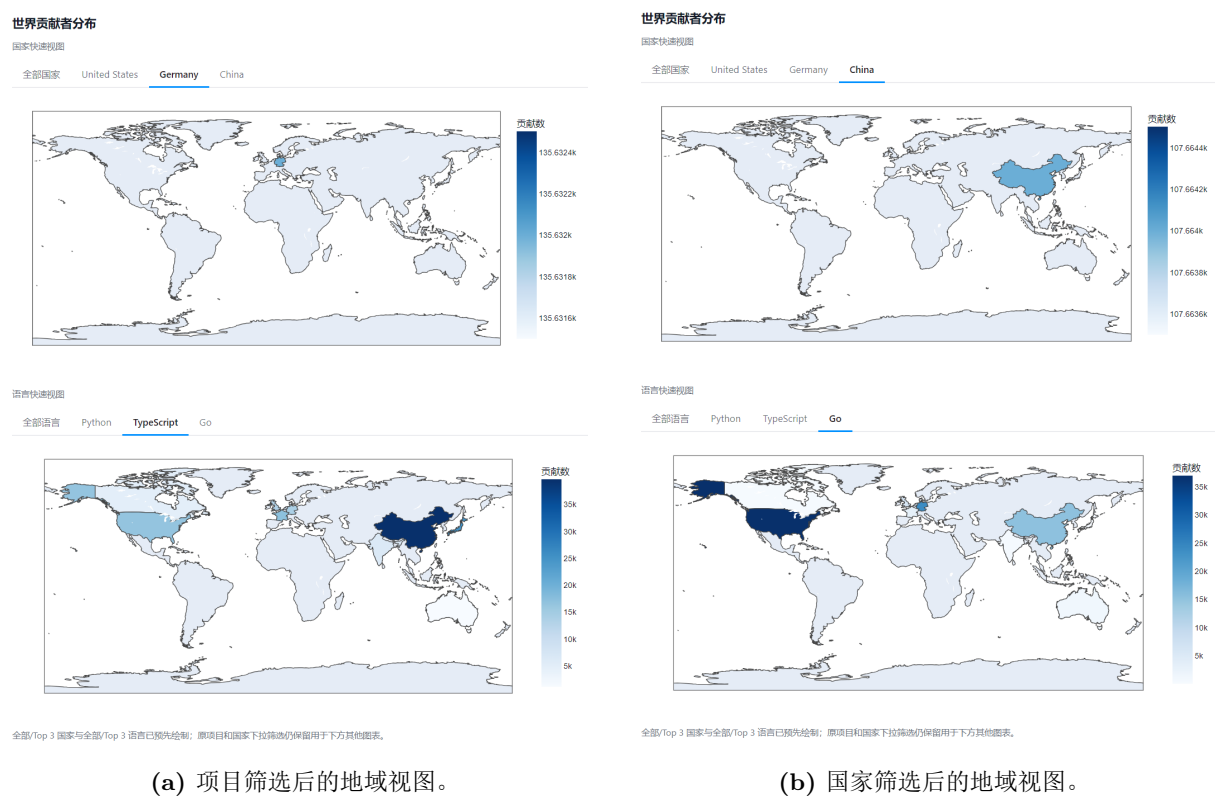
模型输出在这里被进一步转成可解释图表：聚类雷达图概括各类画像，指标热力图显示强弱差异，风险排行则把异常或高风险项目落到具体名单。



(a) 多地图对照视图。

(b) 项目地域分布面板。

图 23: 地域分析补充图表截图二



(a) 项目筛选后的地域视图。

(b) 国家筛选后的地域视图。

图 24: 地域页交互筛选补充截图

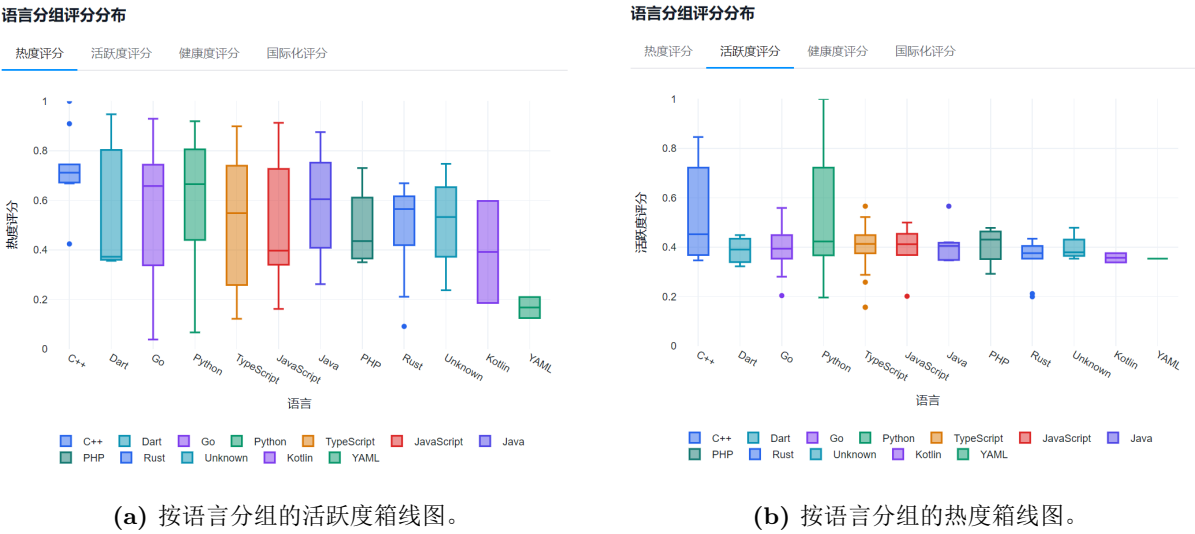


图 25: 统计页语言分组评分补充截图一

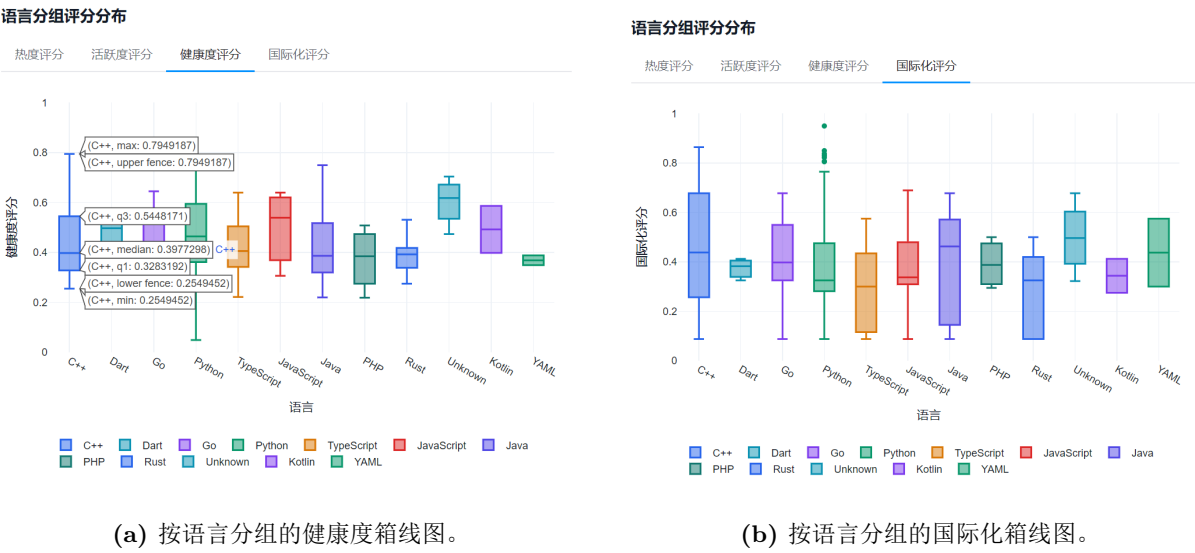


图 26: 统计页语言分组评分补充截图二

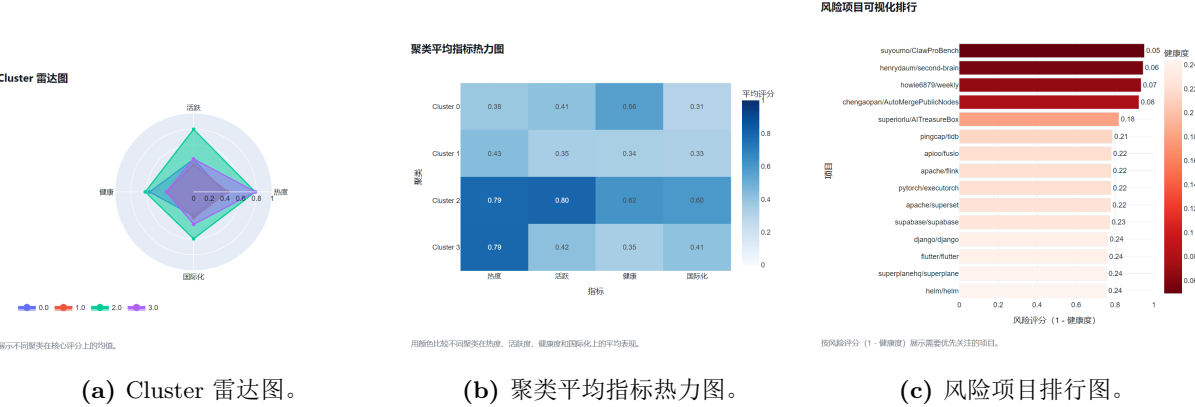


图 27: 机器学习补充图表截图一



(a) 聚类评分柱状图。(b) 模型指标表。(c) 风险项目明细表。

图 28: 机器学习补充图表截图二

聚类评分、模型指标和风险项目明细构成了从整体评估到具体对象的补充链条，有助于把散点图上的分组和异常点重新对应到可检查的项目记录。

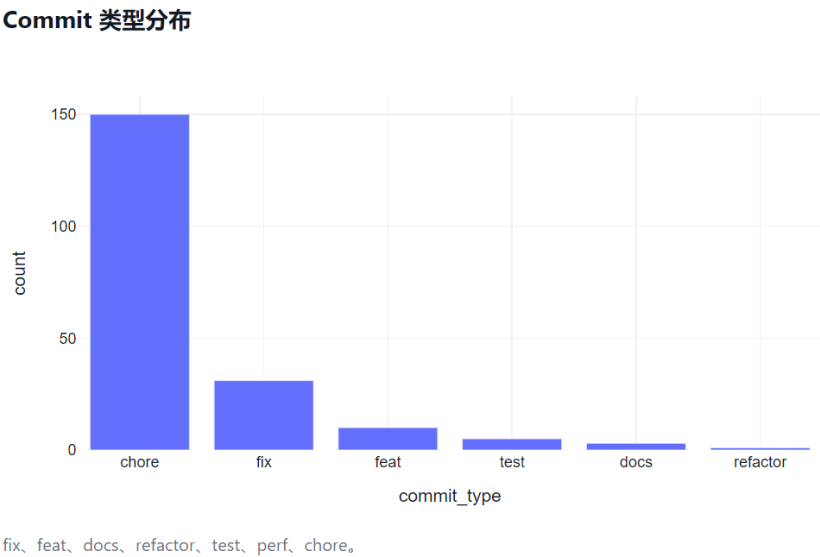


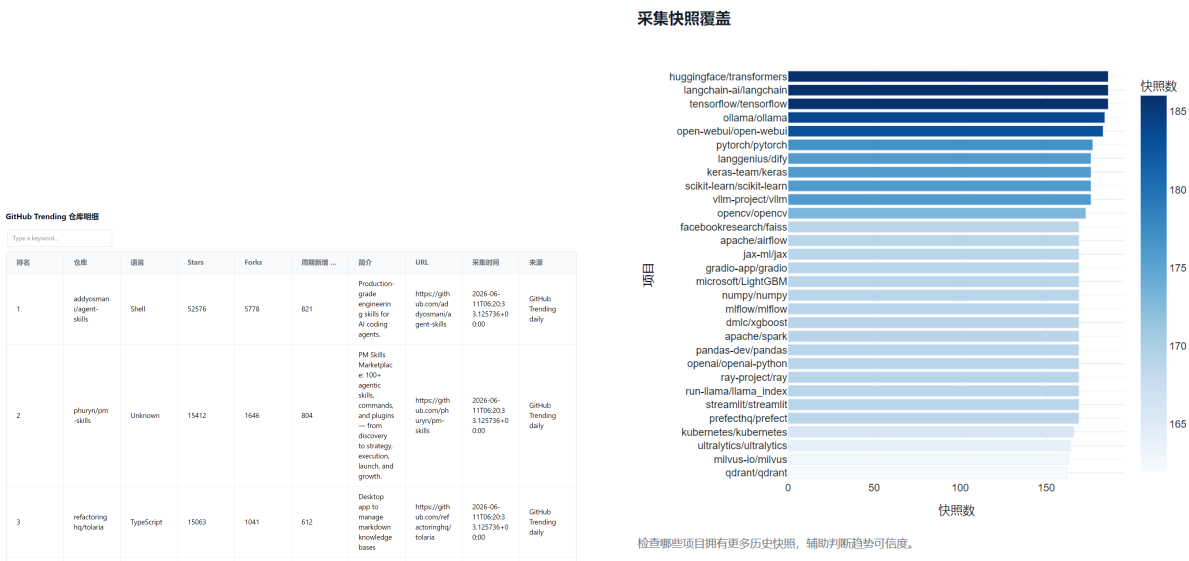
图 29: 文本分析补充图表截图：Commit 类型分布

除 README 语义、项目相似度和 Issue 主题外，Commit 类型分布补充了开发活动结构视角，可以观察修复、功能、文档、测试、重构等提交类型在样本中的占比。

这里补充的是实时榜单和时间趋势的细节层面：Trending 表格提供项目明细，Open Issues 变化排行则从维护压力角度解释时间窗口内的变化。

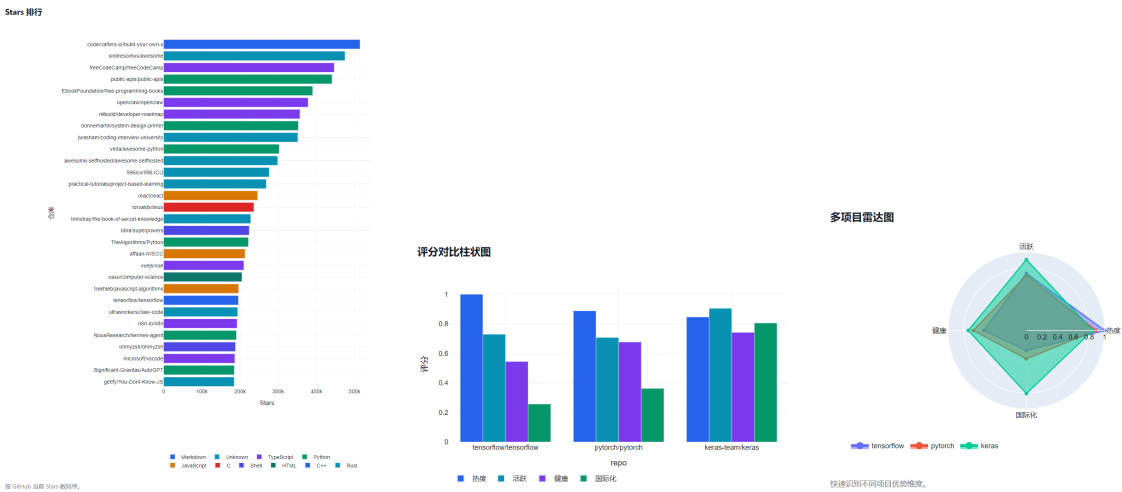
Stars 榜与项目对比图放在一起，可以把“谁更受关注”和“多维评分是否均衡”联系起来看，避免只根据热度排行判断项目整体状态。

最后的 Issue/PR 明细表把项目对比进一步落到协作过程。通过打开问题、关闭问题和 Pull Request 等字段，可以解释综合评分差异背后的维护行为。



(a) Trending 项目明细表。(b) Open Issues 变化排行图。

图 30: 实时榜与趋势页补充截图



(a) Stars 榜项目排行图。(b) 项目评分对比柱状图。(c) 多项目雷达图。

图 31: Stars 榜与项目对比补充截图一

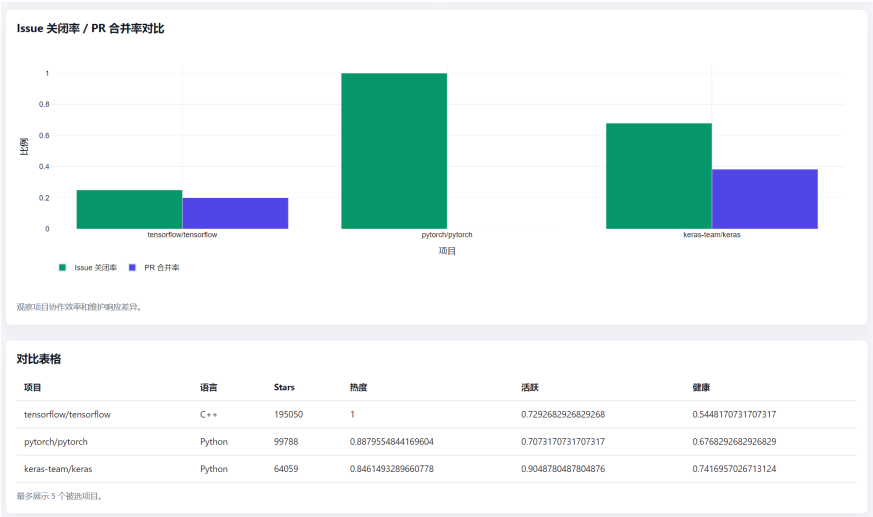


图 32: 项目对比页 Issue/PR 明细表，用于比较多个项目的协作效率和近期维护状态。

参考文献

- [1] GitHub Docs. REST API endpoints for search, 2026. Accessed: 2026-06-10. URL: <https://docs.github.com/en/rest/search/search>.
- [2] GitHub Docs. REST API endpoints for repositories, 2026. Accessed: 2026-06-10. URL: <https://docs.github.com/en/rest/repos/repos>.
- [3] GitHub. GitHub Trending, 2026. Accessed: 2026-06-10. URL: <https://github.com/trending>.
- [4] Reflex. Reflex Documentation: Introduction, 2026. Accessed: 2026-06-10. URL: <https://reflex.dev/docs/getting-started/introduction/>.
- [5] Plotly. Plotly Python Graphing Library, 2026. Accessed: 2026-06-10. URL: <https://plotly.com/python/>.
- [6] CHAOSS. Metric: Project Popularity, 2026. Accessed: 2026-06-10. URL: <https://chaoss.community/kb/metric-project-popularity/>.
- [7] CHAOSS. Metric: Project Velocity, 2026. Accessed: 2026-06-10. URL: <https://chaoss.community/kb/metric-project-velocity/>.
- [8] CHAOSS. Metric: Contributors, 2026. Accessed: 2026-06-10. URL: <https://chaoss.community/kb/metric-contributors/>.
- [9] CHAOSS. Metric: Contributor Location, 2026. Accessed: 2026-06-10. URL: <https://chaoss.community/kb/metric-contributor-location/>.
- [10] Hudson Borges, Andre Hora, and Marco Tulio Valente. Understanding the Factors That Impact the Popularity of GitHub Repositories. In *2016 IEEE International Conference on Software Maintenance and Evolution (ICSME)*, 2016. doi:10.1109/ICSME.2016.31.
- [11] Yue Zhang, Huaimin Wang, Gang Yin, Yue Yu, and Tao Wang. Pull Request Decision Explained: An Empirical Overview. *arXiv preprint arXiv:2105.13970*, 2021. URL: <https://arxiv.org/abs/2105.13970>.
- [12] CHAOSS. Metric: Change Request Acceptance Ratio, 2026. Accessed: 2026-06-10. URL: <https://chaoss.community/kb/metric-change-request-acceptance-ratio/>.
- [13] CHAOSS. Metric: Code Changes Commits, 2026. Accessed: 2026-06-10. URL: <https://chaoss.community/kb/metric-code-changes-commits/>.

- [14] CHAOSS. Metric: Issues Closed, 2026. Accessed: 2026-06-10. URL: <https://chaoss.community/kb/metric-issues-closed/>.
- [15] pandas development team. pandas documentation, 2026. Accessed: 2026-06-10. URL: <https://pandas.pydata.org/docs/>.
- [16] Fabian Pedregosa, Gael Varoquaux, Alexandre Gramfort, Vincent Michel, Bertrand Thirion, Olivier Grisel, Mathieu Blondel, Peter Prettenhofer, Ron Weiss, Vincent Dubourg, Jake Vanderplas, Alexandre Passos, David Cournapeau, Matthieu Brucher, Matthieu Perrot, and Edouard Duchesnay. Scikit-learn: Machine learning in python. *Journal of Machine Learning Research*, 12:2825–2830, 2011.
- [17] scikit-learn developers. KMeans, 2026. Accessed: 2026-06-10. URL: <https://scikit-learn.org/stable/modules/generated/sklearn.cluster.KMeans.html>.
- [18] scikit-learn developers. PCA, 2026. Accessed: 2026-06-10. URL: <https://scikit-learn.org/stable/modules/generated/sklearn.decomposition.PCA.html>.
- [19] Fei Tony Liu, Kai Ming Ting, and Zhi-Hua Zhou. Isolation forest. In *2008 Eighth IEEE International Conference on Data Mining*, pages 413–422, 2008. doi:10.1109/ICDM.2008.17.
- [20] scikit-learn developers. IsolationForest, 2026. Accessed: 2026-06-10. URL: <https://scikit-learn.org/stable/modules/generated/sklearn.ensemble.IsolationForest.html>.